ACID Properties in Databases

Ensuring Data Integrity and Transaction Reliability

What is a Transaction?

A transaction is a single logical unit of work that accesses and possibly modifies the contents of a database. To maintain integrity, every transaction must follow the ACID model.

A - Atomicity ("All or Nothing")

A transaction consists of multiple steps. Atomicity ensures that either **all** steps are committed successfully, or **none** of them are. If any part fails, the entire transaction is "rolled back."

- *Example:* In a bank transfer, deducting money from one account and adding it to another must happen together. If the system crashes after deduction but before addition, the money is returned to the sender.

C - Consistency ("Valid State Only")

Consistency ensures that a transaction takes the database from one valid state to another, maintaining all predefined rules (constraints, cascades, triggers).

- *Example:* If an account has a "Minimum Balance" constraint of $0, a transaction that would result in a -$10 balance will be rejected to maintain consistency.

I - Isolation ("Independent Execution")

Isolation ensures that concurrently executing transactions do not interfere with each other. Each transaction should feel like it is the only one running on the system.

- *Example:* If two people deposit $50 into the same account simultaneously, isolation ensures the final balance increases by exactly $100, rather than one deposit overwriting the other.

D - Durability ("Permanent Record")

Once a transaction has been committed, it will remain so, even in the event of a system failure (e.g., power outage or crash). The results are permanently stored in non-volatile memory.

- *Example:* Once you see the "Success" screen for a flight booking, your seat is reserved even if the airline's database server restarts immediately afterward.

Summary Table: The ACID Test

Property	**Key Word**
Atomicity	Completeness
Consistency	Correctness
Isolation	Independence
Durability	Permanence